Coding Problem 1 : The Interstellar Mining Fleet Manager

Scenario Background:

You are a software engineer for the *Orion Mining Consortium*. You have been tasked with building the onboard memory-management system for a fleet of mining spaceships. Because the onboard computers on these ships have severely limited RAM, you must be extremely deliberate with your choice of **Data Types**. You cannot just use `int` and `double` for everything; you must use the most memory-efficient primitive data types available.

Furthermore, the storage bays on these ships are physically laid out in a grid, meaning the cargo tracking system must be implemented using specific **Array** structures.

Implementation Constraints & Requirements:

1. Data Type Strictness: You must choose the exact primitive data type that fits the maximum possible value with the least memory footprint:

- **Ship ID:** The consortium will never have more than `30,000` ships.
- **Bay Number:** A ship has a maximum of `100` storage bays.
- **Operational Status:** A ship is either actively mining or it is not.
- **Fleet Classification:** A single character representing the ship class ('A', 'B', or 'C').
- **Ore Weight:** The weight of an ore container can have decimal values up to `50,000.00` kg. High-precision decimals are not required, but memory saving is.
- **Total Fleet Value:** The total value of all mined ore across the fleet can exceed `40 Billion` credits (whole numbers only).

2. Inheritance Structure:

- Create an abstract base class called `SpaceVessel` that holds the Ship ID, Operational Status, and Fleet Classification using the strict data types defined above.
- Create a derived class called `MiningShip` that `extends SpaceVessel`.

3. Array Structures:

- Inside `MiningShip`, implement the cargo hold as a **2D Array** of the ore weights. The rows represent the *Bays*, and the columns represent the *Containers* within that bay.
- In your main system, maintain the entire fleet using a **1D Array of Objects** (`SpaceVessel[]`).

4. Required Methods in `MiningShip`:

- `public [datatype] calculateTotalOreWeight()`: Must iterate through the 2D array and return the sum of all ore weights on the ship.
- `public [datatype] findHeaviestContainer()`: Must traverse the 2D array and return the exact weight of the heaviest single container.

Coding Problem 2: The "Colony Grid" Power Manager (Bitwise Mechanics)

Scenario Background:

You are programming the power distribution node for an off-grid Mars colony. A single node controls exactly 8 distinct power sectors (e.g., Life Support, Hydroponics, Communications).

Memory is critically limited on the micro-controllers. A junior developer tried to use a boolean[] of size 8 to track if the sectors are ON or OFF. However, arrays carry object header overhead in the JVM, wasting precious memory. You must refactor the system to track the ON/OFF state of all 8 sectors using a **single primitive variable**, utilizing Bitwise operators to read and manipulate the states.

Implementation Constraints & Requirements:

1. Data Type Strictness:

- You must use a single byte (8 bits) named sectorStates to store the ON/OFF status of all 8 sectors.
- Sector 0 is the least significant bit (rightmost), and Sector 7 is the most significant bit (leftmost).

2. Bitwise Operations:

- public void turnOnSector(int sectorIndex): Must use the Bitwise OR (|) and Left Shift (<<) operators to turn the specific bit to 1.
- public void turnOffSector(int sectorIndex): Must use Bitwise AND (&), Bitwise NOT (~), and Left Shift (<<) to turn the specific bit to 0.
- public boolean isSectorOn(int sectorIndex): Must use Bitwise AND (&) and Left Shift (<<) to check if a specific bit is 1.

Coding Problem 3: The "DNA Sequencer" Memory Leak (String Immutability)

Scenario Background:

You are analyzing alien DNA sequences for a bio-tech lab. A sequence is represented by a massive string of characters ('A', 'C', 'T', 'G').

A previous developer wrote a loop that reads 100,000 characters from a sensor and appends them to a standard String variable using the += operator. The application keeps crashing with an OutOfMemoryError due to severe Garbage Collection (GC) overhead. You must redesign the sequence builder to be highly memory-efficient and prevent string pool bloat.

Implementation Constraints & Requirements:

1. Object Management Strictness:

- You are strictly forbidden from using the += operator or the .concat() method on String objects inside your loops.
- You must use the StringBuilder class to handle all character appends.
- You must instantiate the StringBuilder with an **initial capacity** to prevent the underlying char array from constantly resizing itself.

2. Required Methods:

- public void ingestSequence(char[] sensorData): Must efficiently append an array of characters.
- public void mutateDNA(String target, String replacement): Must find the first occurrence of a specific sequence and replace it *in-place* without creating new String objects if possible.

Coding Problem 4: The "Deep-Sea Telemetry" Error Hierarchy (Checked vs. Unchecked)

Scenario Background:

You are writing the parser for an autonomous deep-sea submarine. The sub writes telemetry data to a local file. Sometimes the file is locked by the OS, sometimes the sensors report impossible physical values (like a water temperature of 500°C), and sometimes the pressure sensor simply disconnects.

You need to build a robust error-handling hierarchy that differentiates between **fatal hardware failures** (which should crash the specific process) and **recoverable sensor glitches** (which should just be logged).

Implementation Constraints & Requirements:

1. Exception Hierarchy:

- Create a custom *Checked Exception* called HardwareLockException. (Requires method signatures to declare throws).
- Create a custom *Unchecked Exception* called SensorCorruptionException. (Does not require throws declarations).

2. Try-With-Resources:

- Create a dummy class TelemetryStream that implements AutoCloseable.
- In your main parser method, you must use a **Try-With-Resources** block to ensure the TelemetryStream is always closed, even if an exception is thrown, completely eliminating the need for a finally block.

Coding Problem 5: The "Drone Hive" Synchronization (Volatile & Atomics)

Scenario Background:

You are coordinating a swarm of 10,000 delivery drones returning to a central Hive. Every time a drone lands, it increments a global totalDronesReturned counter.

Simultaneously, a central radar scans for storms. If a storm is detected, an emergencyAbort flag is set to true, and all drones currently in the air must instantly alter their route. Because 10,000 separate threads (drones) are accessing these two variables at the exact same time, you are experiencing severe **Race Conditions** and **Memory Visibility** issues.

Implementation Constraints & Requirements:

1. Concurrency Strictness:

- You cannot use the synchronized keyword, as locking the entire method will bottleneck the 10,000 drones and crash the scheduling system.
- You must use the AtomicInteger class to ensure the returned drone counter increments safely without race conditions.
- You must use the volatile keyword on the emergencyAbort boolean to guarantee that the moment the central radar changes the flag, all 10,000 threads instantly see the updated value, bypassing local thread CPU caching.

Problem 1: The AWS S3 Asymmetric Video Chunking Buffer (Jagged Arrays)

Enterprise Scenario: When streaming massive video files to an AWS S3 bucket, backends use byte arrays to stream data in chunks. However, due to network fluctuations, the chunks arrive in non-uniform sizes.

Task: Write a program that generates a jagged 2D array where the number of rows is 5, but the number of columns for each row corresponds to the first 5 numbers of the Fibonacci sequence (1, 1, 2, 3, 5). Populate the matrix with sequential integers. Finally, calculate the total sum of all elements without triggering an ArrayIndexOutOfBoundsException.

Problem 2: High-Frequency Trading Truncation Defect (Primitive Casting)

Enterprise Scenario: A high-frequency trading platform calculates aggregate transaction totals in double for precision. A legacy downstream system requires the final payload size in byte to optimize network transmission.

Task: Write a program that takes an initial transaction value of 130.99 (double). Cast it to an int to drop the fractional cents, and then cast that result to a byte for the legacy system. Predict and print the exact outputs, explaining the mathematical defect caused by the JVM.

Problem 3: The Pass-by-Value Reference Trap (Stack vs. Heap)

Enterprise Scenario: A junior developer is tasked with writing a security method that wipes sensitive user data by reassigning the array reference to a new, empty array. However, the sensitive data is still leaking in production.

Task: Write a program that passes an int[] to a helper method. In the helper method, attempt to reassign the array to new int[3]. Print the array in the main method afterward to prove the data was not wiped. Then, write a second helper method that correctly wipes the data by mutating the indices directly.

Problem 4: In-Place Image Rotation (Algorithmic Array Manipulation)

Enterprise Scenario: An image processing microservice represents pixel data as a 2D square matrix (N x N). To optimize memory, you cannot create a secondary array; you must rotate the image 90 degrees clockwise *in-place*.

Task: Write a method that accepts a 3 x 3 2D array. Transpose the matrix (swap rows with columns) and then reverse each row to achieve a 90-degree clockwise rotation, modifying the original heap allocation directly.

Problem 5: The Garbage Collector Memory Game (Reference Severing)

Enterprise Scenario: A backend cache maintains large array buffers. If the references to these arrays are not managed correctly, the Garbage Collector cannot reclaim the RAM, leading to memory exhaustion.

Task: Write a program that creates three distinct arrays in the heap. Intersect their reference pointers (e.g., store one array inside a 2D array). Then, systematically severe the references. Add inline comments predicting the exact moment a specific array payload becomes "unreachable" and eligible for Garbage Collection.